

11 PLANNING

In which we see how an agent can take advantage of the structure of a problem to construct complex plans of action.

The task of coming up with a sequence of actions that will achieve a goal is called **planning**. We have seen two examples of planning agents so far: the search-based problem-solving agent of Chapter 3 and the logical planning agent of Chapter 10. This chapter is concerned primarily with *scaling up* to complex planning problems that defeat the approaches we have seen so far.

Section 11.1 develops an expressive yet carefully constrained language for representing planning problems, including actions and states. The language is closely related to the propositional and first-order representations of actions in Chapters 7 and 10. Section 11.2 shows how forward and backward search algorithms can take advantage of this representation, primarily through accurate heuristics that can be derived automatically from the structure of the representation. (This is analogous to the way in which effective heuristics were constructed for constraint satisfaction problems in Chapter 5.) Sections 11.3 through 11.5 describe planning algorithms that go beyond forward and backward search, taking advantage of the representation of the problem. In particular, we explore approaches that are not constrained to consider only totally ordered sequences of actions.

For this chapter, we consider only environments that are fully observable, deterministic, finite, static (change happens only when the agent acts), and discrete (in time, action, objects, and effects). These are called **classical planning** environments. In contrast, nonclassical planning is for partially observable or stochastic environments and involves a different set of algorithms and agent designs, outlined in Chapters 12 and 17.

CLASSICAL
PLANNING

11.1 THE PLANNING PROBLEM

Let us consider what can happen when an ordinary problem-solving agent using standard search algorithms—depth-first, A^* , and so on—comes up against large, real-world problems. That will help us design better planning agents.

The most obvious difficulty is that the problem-solving agent can be overwhelmed by irrelevant actions. Consider the task of buying a copy of *AI: A Modern Approach* from an online bookseller. Suppose there is one buying action for each 10-digit ISBN number, for a total of 10 billion actions. The search algorithm would have to examine the outcome states of all 10 billion actions to find one that satisfies the goal, which is to own a copy of ISBN 0137903952. A sensible planning agent, on the other hand, should be able to work back from an explicit goal description such as $Have(ISBN0137903952)$ and generate the action $Buy(ISBN0137903952)$ directly. To do this, the agent simply needs the general knowledge that $Buy(x)$ results in $Have(x)$. Given this knowledge and the goal, the planner can decide in a single unification step that $Buy(ISBN0137903952)$ is the right action.

The next difficulty is finding a good **heuristic function**. Suppose the agent's goal is to buy four different books online. Then there will be 10^{40} plans of just four steps, so searching without an accurate heuristic is out of the question. It is obvious to a human that a good heuristic estimate for the cost of a state is the number of books that remain to be bought; unfortunately, this insight is not obvious to a problem-solving agent, because it sees the goal test only as a black box that returns true or false for each state. Therefore, the problem-solving agent lacks autonomy; it requires a human to supply a heuristic function for each new problem. On the other hand, if a planning agent has access to an explicit representation of the goal as a conjunction of subgoals, then it can use a single *domain-independent* heuristic: the number of unsatisfied conjuncts. For the book-buying problem, the goal would be $Have(A) \wedge Have(B) \wedge Have(C) \wedge Have(D)$, and a state containing $Have(A) \wedge Have(C)$ would have cost 2. Thus, the agent automatically gets the right heuristic for this problem, and for many others. We shall see later in the chapter how to construct more sophisticated heuristics that examine the available actions as well as the structure of the goal.

PROBLEM DECOMPOSITION

Finally, the problem solver might be inefficient because it cannot take advantage of **problem decomposition**. Consider the problem of delivering a set of overnight packages to their respective destinations, which are scattered across Australia. It makes sense to find out the nearest airport for each destination and divide the overall problem into several subproblems, one for each airport. Within the set of packages routed through a given airport, whether further decomposition is possible depends on the destination city. We saw in Chapter 5 that the ability to do this kind of decomposition contributes to the efficiency of constraint satisfaction problem solvers. The same holds true for planners: in the worst case, it can take $O(n!)$ time to find the best plan to deliver n packages, but only $O((n/k)! \times k)$ time if the problem can be decomposed into k equal parts.

NEARLY DECOMPOSABLE

As we noted in Chapter 5, perfectly decomposable problems are delicious but rare.¹ The design of many planning systems—particularly the partial-order planners described in Section 11.3—is based on the assumption that most real-world problems are **nearly decomposable**. That is, the planner can work on subgoals independently, but might need to do some additional work to combine the resulting subplans. For some problems, this assump-

¹ Notice that even the delivery of a package is not perfectly decomposable. There may be cases in which it is better to assign packages to a more distant airport if that renders a flight to the nearest airport unnecessary. Nevertheless, most delivery companies prefer the computational and organizational simplicity of sticking with decomposed solutions.

tion breaks down because working on one subgoal is likely to undo another subgoal. These interactions among subgoals are what makes puzzles (like the 8-puzzle) puzzling.

The language of planning problems

The preceding discussion suggests that the representation of planning problems—states, actions, and goals—should make it possible for planning algorithms to take advantage of the logical structure of the problem. The key is to find a language that is expressive enough to describe a wide variety of problems, but restrictive enough to allow efficient algorithms to operate over it. In this section, we first outline the basic representation language of classical planners, known as the STRIPS language.² Later, we point out some of the many possible variations in STRIPS-like languages.

Representation of states. Planners decompose the world into logical conditions and represent a state as a conjunction of positive literals. We will consider propositional literals; for example, $Poor \wedge Unknown$ might represent the state of a hapless agent. We will also use first-order literals; for example, $At(Plane_1, Melbourne) \wedge At(Plane_2, Sydney)$ might represent a state in the package delivery problem. Literals in first-order state descriptions must be **ground** and **function-free**. Literals such as $At(x, y)$ or $At(Father(Fred), Sydney)$ are not allowed. The **closed-world assumption** is used, meaning that any conditions that are not mentioned in a state are assumed false.

Representation of goals. A goal is a partially specified state, represented as a conjunction of positive ground literals, such as $Rich \wedge Famous$ or $At(P_2, Tahiti)$. A propositional state s **satisfies** a goal g if s contains all the atoms in g (and possibly others). For example, the state $Rich \wedge Famous \wedge Miserable$ satisfies the goal $Rich \wedge Famous$.

Representation of actions. An action is specified in terms of the preconditions that must hold before it can be executed and the effects that ensue when it is executed. For example, an action for flying a plane from one location to another is:

$$\begin{aligned} &Action(Fly(p, from, to), \\ &\quad PRECOND: At(p, from) \wedge Plane(p) \wedge Airport(from) \wedge Airport(to) \\ &\quad EFFECT: \neg At(p, from) \wedge At(p, to)) \end{aligned}$$

ACTION SCHEMA

This is more properly called an **action schema**, meaning that it represents a number of different actions that can be derived by instantiating the variables p , $from$, and to to different constants. In general, an action schema consists of three parts:

- The action name and parameter list—for example, $Fly(p, from, to)$ —serves to identify the action.
- The **precondition** is a conjunction of function-free positive literals stating what must be true in a state before the action can be executed. Any variables in the precondition must also appear in the action's parameter list.
- The **effect** is a conjunction of function-free literals describing how the state changes when the action is executed. A positive literal P in the effect is asserted to be true in

PRECONDITION

EFFECT

² STRIPS stands for STanford Research Institute Problem Solver.

the state resulting from the action, whereas a negative literal $\neg P$ is asserted to be false. Variables in the effect must also appear in the action's parameter list.

ADD LIST
DELETE LIST

To improve readability, some planning systems divide the effect into the **add list** for positive literals and the **delete list** for negative literals.

APPLICABLE

Having defined the syntax for representations of planning problems, we can now define the semantics. The most straightforward way to do this is to describe how actions affect states. (An alternative method is to specify a direct translation into successor-state axioms, whose semantics comes from first-order logic; see Exercise 11.3.) First, we say that an action is **applicable** in any state that satisfies the precondition; otherwise, the action has no effect. For a first-order action schema, establishing applicability will involve a substitution θ for the variables in the precondition. For example, suppose the current state is described by

$$At(P_1, JFK) \wedge At(P_2, SFO) \wedge Plane(P_1) \wedge Plane(P_2) \\ \wedge Airport(JFK) \wedge Airport(SFO) .$$

This state satisfies the precondition

$$At(p, from) \wedge Plane(p) \wedge Airport(from) \wedge Airport(to)$$

with substitution $\{p/P_1, from/JFK, to/SFO\}$ (among others—see Exercise 11.2). Thus, the concrete action $Fly(P_1, JFK, SFO)$ is applicable.

RESULT

Starting in state s , the **result** of executing an applicable action a is a state s' that is the same as s except that any positive literal P in the effect of a is added to s' and any negative literal $\neg P$ is removed from s' . Thus, after $Fly(P_1, JFK, SFO)$, the current state becomes

$$At(P_1, SFO) \wedge At(P_2, SFO) \wedge Plane(P_1) \wedge Plane(P_2) \\ \wedge Airport(JFK) \wedge Airport(SFO) .$$

Note that if a positive effect is already in s it is not added twice, and if a negative effect is not in s , then that part of the effect is ignored. This definition embodies the so-called STRIPS **assumption**: that every literal not mentioned in the effect remains unchanged. In this way, STRIPS avoids the representational **frame problem** described in Chapter 10.

STRIPS ASSUMPTION

SOLUTION

Finally, we can define the **solution** for a planning problem. In its simplest form, this is just an action sequence that, when executed in the initial state, results in a state that satisfies the goal. Later in the chapter, we will allow solutions to be partially ordered sets of actions, provided that every action sequence that respects the partial order is a solution.

Expressiveness and extensions

The various restrictions imposed by the STRIPS representation were chosen in the hope of making planning algorithms simpler and more efficient, without making it too difficult to describe real problems. One of the most important restrictions is that literals be *function-free*. With this restriction, we can be sure that any action schema for a given problem can be propositionalized—that is, turned into a finite collection of purely propositional action representations with no variables. (See Chapter 9 for more on this topic.) For example, in the air cargo domain for a problem with 10 planes and five airports, we could translate the $Fly(p, from, to)$ schema into $10 \times 5 \times 5 = 250$ purely propositional actions. The planners

STRIPS Language	ADL Language
Only positive literals in states: $Poor \wedge Unknown$	Positive and negative literals in states: $\neg Rich \wedge \neg Famous$
Closed World Assumption: Unmentioned literals are false.	Open World Assumption: Unmentioned literals are unknown.
Effect $P \wedge \neg Q$ means add P and delete Q .	Effect $P \wedge \neg Q$ means add P and $\neg Q$ and delete $\neg P$ and Q .
Only ground literals in goals: $Rich \wedge Famous$	Quantified variables in goals: $\exists x At(P_1, x) \wedge At(P_2, x)$ is the goal of having P_1 and P_2 in the same place.
Goals are conjunctions: $Rich \wedge Famous$	Goals allow conjunction and disjunction: $\neg Poor \wedge (Famous \vee Smart)$
Effects are conjunctions.	Conditional effects allowed: when P : E means E is an effect only if P is satisfied.
No support for equality.	Equality predicate ($x = y$) is built in.
No support for types.	Variables can have types, as in ($p : Plane$).
Figure 11.1 Comparison of STRIPS and ADL languages for representing planning problems. In both cases, goals behave as the preconditions of an action with no parameters.	

in Sections 11.4 and 11.5 work directly with propositionalized descriptions. If we allow function symbols, then infinitely many states and actions can be constructed.

In recent years, it has become clear that STRIPS is insufficiently expressive for some real domains. As a result, many language variants have been developed. Figure 11.1 briefly describes one important one, the Action Description Language or **ADL**, by comparing it with the basic STRIPS language. In ADL, the *Fly* action could be written as

Action(*Fly*($p : Plane, from : Airport, to : Airport$),
 PRECOND: $At(p, from) \wedge (from \neq to)$
 EFFECT: $\neg At(p, from) \wedge At(p, to)$).

The notation $p : Plane$ in the parameter list is an abbreviation for $Plane(p)$ in the precondition; this adds no expressive power, but can be easier to read. (It also cuts down on the number of possible propositional actions that can be constructed.) The precondition ($from \neq to$) expresses the fact that a flight cannot be made from an airport to itself. This could not be expressed succinctly in STRIPS.

The various planning formalisms used in AI have been systematized within a standard syntax called the the Planning Domain Definition Language, or PDDL. This language allows researchers to exchange benchmark problems and compare results. PDDL includes sublanguages for STRIPS, ADL, and the hierarchical task networks we will see in Chapter 12.

```

Init(At(C1, SFO) ∧ At(C2, JFK) ∧ At(P1, SFO) ∧ At(P2, JFK)
    ∧ Cargo(C1) ∧ Cargo(C2) ∧ Plane(P1) ∧ Plane(P2)
    ∧ Airport(JFK) ∧ Airport(SFO))
Goal(At(C1, JFK) ∧ At(C2, SFO))
Action(Load(c, p, a),
    PRECOND: At(c, a) ∧ At(p, a) ∧ Cargo(c) ∧ Plane(p) ∧ Airport(a)
    EFFECT: ¬ At(c, a) ∧ In(c, p))
Action(Unload(c, p, a),
    PRECOND: In(c, p) ∧ At(p, a) ∧ Cargo(c) ∧ Plane(p) ∧ Airport(a)
    EFFECT: At(c, a) ∧ ¬ In(c, p))
Action(Fly(p, from, to),
    PRECOND: At(p, from) ∧ Plane(p) ∧ Airport(from) ∧ Airport(to)
    EFFECT: ¬ At(p, from) ∧ At(p, to))

```

Figure 11.2 A STRIPS problem involving transportation of air cargo between airports.

The STRIPS and ADL notations are adequate for many real domains. The subsections that follow show some simple examples. There are still some significant restrictions, however. The most obvious is that they cannot represent in a natural way the **ramifications** of actions. For example, if there are people, packages, or dust motes in the airplane, then they too change location when the plane flies. We can represent these changes as the direct effects of flying, whereas it seems more natural to represent the location of the plane's contents as a logical consequence of the location of the plane. We will see more examples of such **state constraints** in Section 11.5. Classical planning systems do not even attempt to address the **qualification** problem: the problem of unrepresented circumstances that could cause an action to fail. We will see how to address qualifications in Chapter 12.

STATE CONSTRAINTS

Example: Air cargo transport

Figure 11.2 shows an air cargo transport problem involving loading and unloading cargo onto and off of planes and flying it from place to place. The problem can be defined with three actions: *Load*, *Unload*, and *Fly*. The actions affect two predicates: *In*(*c*, *p*) means that cargo *c* is inside plane *p*, and *At*(*x*, *a*) means that object *x* (either plane or cargo) is at airport *a*. Note that cargo is not *At* anywhere when it is *In* a plane, so *At* really means “available for use at a given location.” It takes some experience with action definitions to handle such details consistently. The following plan is a solution to the problem:

```

[Load(C1, P1, SFO), Fly(P1, SFO, JFK),
 Load(C2, P2, JFK), Fly(P2, JFK, SFO)] .

```

Our representation is pure STRIPS. In particular, it allows a plane to fly to and from the same airport. Inequality literals in ADL could prevent this.

Example: The spare tire problem

Consider the problem of changing a flat tire. More precisely, the goal is to have a good spare tire properly mounted onto the car's axle, where the initial state has a flat tire on the axle and a good spare tire in the trunk. To keep it simple, our version of the problem is a very abstract one, with no sticky lug nuts or other complications. There are just four actions: removing the spare from the trunk, removing the flat tire from the axle, putting the spare on the axle, and leaving the car unattended overnight. We assume that the car is in a particularly bad neighborhood, so that the effect of leaving it overnight is that the tires disappear.

The ADL description of the problem is shown in Figure 11.3. Notice that it is purely propositional. It goes beyond STRIPS in that it uses a negated precondition, $\neg At(Flat, Axle)$, for the *PutOn*(*Spare*, *Axle*) action. This could be avoided by using *Clear*(*Axle*) instead, as we will see in the next example.

```

Init(At(Flat, Axle)  $\wedge$  At(Spare, Trunk))
Goal(At(Spare, Axle))
Action(Remove(Spare, Trunk),
  PRECOND: At(Spare, Trunk)
  EFFECT:  $\neg At$ (Spare, Trunk)  $\wedge$  At(Spare, Ground))
Action(Remove(Flat, Axle),
  PRECOND: At(Flat, Axle)
  EFFECT:  $\neg At$ (Flat, Axle)  $\wedge$  At(Flat, Ground))
Action(PutOn(Spare, Axle),
  PRECOND: At(Spare, Ground)  $\wedge$   $\neg At$ (Flat, Axle)
  EFFECT:  $\neg At$ (Spare, Ground)  $\wedge$  At(Spare, Axle))
Action(LeaveOvernight,
  PRECOND:
  EFFECT:  $\neg At$ (Spare, Ground)  $\wedge$   $\neg At$ (Spare, Axle)  $\wedge$   $\neg At$ (Spare, Trunk)
         $\wedge$   $\neg At$ (Flat, Ground)  $\wedge$   $\neg At$ (Flat, Axle))

```

Figure 11.3 The simple spare tire problem.

Example: The blocks world

BLOCKS WORLD

One of the most famous planning domains is known as the **blocks world**. This domain consists of a set of cube-shaped blocks sitting on a table.³ The blocks can be stacked, but only one block can fit directly on top of another. A robot arm can pick up a block and move it to another position, either on the table or on top of another block. The arm can pick up only one block at a time, so it cannot pick up a block that has another one on it. The goal will always be to build one or more stacks of blocks, specified in terms of what blocks are on top of what other blocks. For example, a goal might be to get block *A* on *B* and block *C* on *D*.

³ The blocks world used in planning research is much simpler than SHRDLU's version, shown on page 20.

We will use $On(b, x)$ to indicate that block b is on x , where x is either another block or the table. The action for moving block b from the top of x to the top of y will be $Move(b, x, y)$. Now, one of the preconditions on moving b is that no other block be on it. In first-order logic, this would be $\neg \exists x On(x, b)$ or, alternatively, $\forall x \neg On(x, b)$. These could be stated as preconditions in ADL. We can stay within the STRIPS language, however, by introducing a new predicate, $Clear(x)$, that is true when nothing is on x .

The action $Move$ moves a block b from x to y if both b and y are clear. After the move is made, x is clear but y is not. A formal description of $Move$ in STRIPS is

Action($Move(b, x, y)$,
 PRECOND: $On(b, x) \wedge Clear(b) \wedge Clear(y)$,
 EFFECT: $On(b, y) \wedge Clear(x) \wedge \neg On(b, x) \wedge \neg Clear(y)$).

Unfortunately, this action does not maintain $Clear$ properly when x or y is the table. When $x = Table$, this action has the effect $Clear(Table)$, but the table should not become clear, and when $y = Table$, it has the precondition $Clear(Table)$, but the table does not have to be clear to move a block onto it. To fix this, we do two things. First, we introduce another action to move a block b from x to the table:

Action($MoveToTable(b, x)$,
 PRECOND: $On(b, x) \wedge Clear(b)$,
 EFFECT: $On(b, Table) \wedge Clear(x) \wedge \neg On(b, x)$).

Second, we take the interpretation of $Clear(b)$ to be “there is a clear space on b to hold a block.” Under this interpretation, $Clear(Table)$ will always be true. The only problem is that nothing prevents the planner from using $Move(b, x, Table)$ instead of $MoveToTable(b, x)$. We could live with this problem—it will lead to a larger-than-necessary search space, but will not lead to incorrect answers—or we could introduce the predicate $Block$ and add $Block(b) \wedge Block(y)$ to the precondition of $Move$.

Finally, there is the problem of spurious actions such as $Move(B, C, C)$, which should be a no-op, but which has contradictory effects. It is common to ignore such problems, because they seldom cause incorrect plans to be produced. The correct approach is add inequality preconditions as shown in Figure 11.4.

11.2 PLANNING WITH STATE-SPACE SEARCH

Now we turn our attention to planning algorithms. The most straightforward approach is to use state-space search. Because the descriptions of actions in a planning problem specify both preconditions and effects, it is possible to search in either direction: either forward from the initial state or backward from the goal, as shown in Figure 11.5. We can also use the explicit action and goal representations to derive effective heuristics automatically.

Forward state-space search

Planning with forward state-space search is similar to the problem-solving approach of Chapter 3. It is sometimes called **progression** planning, because it moves in the forward direction.

```

Init( $On(A, Table) \wedge On(B, Table) \wedge On(C, Table)$ 
 $\wedge Block(A) \wedge Block(B) \wedge Block(C)$ 
 $\wedge Clear(A) \wedge Clear(B) \wedge Clear(C)$ )
Goal( $On(A, B) \wedge On(B, C)$ )
Action(Move( $b, x, y$ ),
  PRECOND:  $On(b, x) \wedge Clear(b) \wedge Clear(y) \wedge Block(b) \wedge$ 
 $(b \neq x) \wedge (b \neq y) \wedge (x \neq y)$ ,
  EFFECT:  $On(b, y) \wedge Clear(x) \wedge \neg On(b, x) \wedge \neg Clear(y)$ )
Action(MoveToTable( $b, x$ ),
  PRECOND:  $On(b, x) \wedge Clear(b) \wedge Block(b) \wedge (b \neq x)$ ,
  EFFECT:  $On(b, Table) \wedge Clear(x) \wedge \neg On(b, x)$ )

```

Figure 11.4 A planning problem in the blocks world: building a three-block tower. One solution is the sequence [$Move(B, Table, C)$, $Move(A, Table, B)$].

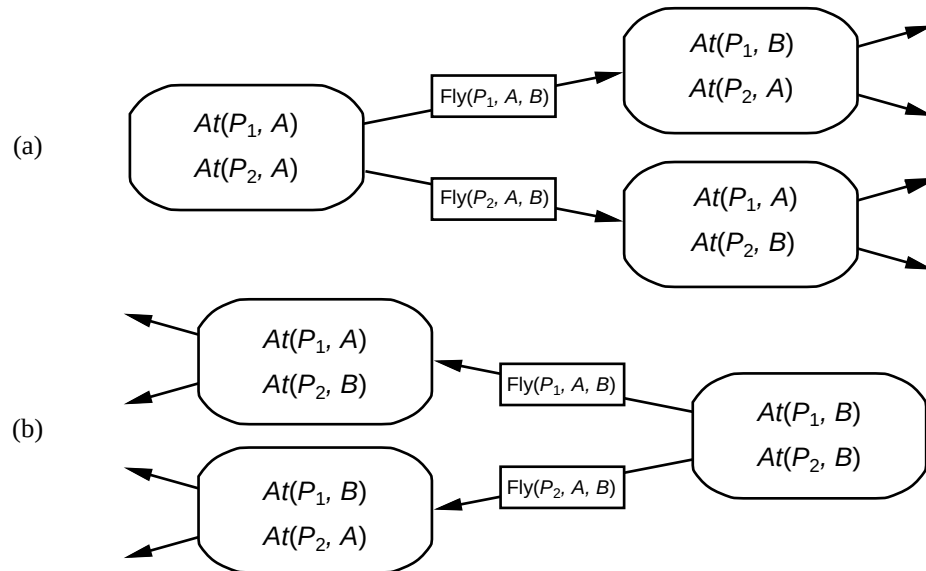


Figure 11.5 Two approaches to searching for a plan. (a) Forward (progression) state-space search, starting in the initial state and using the problem's actions to search forward for the goal state. (b) Backward (regression) state-space search: a belief-state search (see page 84) starting at the goal state(s) and using the inverse of the actions to search backward for the initial state.

We start in the problem's initial state, considering sequences of actions until we find a sequence that reaches a goal state. The formulation of planning problems as state-space search problems is as follows:

- The **initial state** of the search is the initial state from the planning problem. In general, each state will be a set of positive ground literals; literals not appearing are false.

- The **actions** that are applicable to a state are all those whose preconditions are satisfied. The successor state resulting from an action is generated by adding the positive effect literals and deleting the negative effect literals. (In the first-order case, we must apply the unifier from the preconditions to the effect literals.) Note that a single successor function works for all planning problems—a consequence of using an explicit action representation.
- The **goal test** checks whether the state satisfies the goal of the planning problem.
- The **step cost** of each action is typically 1. Although it would be easy to allow different costs for different actions, this is seldom done by STRIPS planners.

Recall that, in the absence of function symbols, the state space of a planning problem is finite. Therefore, any graph search algorithm that is complete—for example, A^* —will be a complete planning algorithm.

From the earliest days of planning research (around 1961) until recently (around 1998) it was assumed that forward state-space search was too inefficient to be practical. It is not hard to come up with reasons why—just refer back to the start of Section 11.1. First, forward search does not address the irrelevant action problem—all applicable actions are considered from each state. Second, the approach quickly bogs down without a good heuristic. Consider an air cargo problem with 10 airports, where each airport has 5 planes and 20 pieces of cargo. The goal is to move all the cargo at airport A to airport B . There is a simple solution to the problem: load the 20 pieces of cargo into one of the planes at A , fly the plane to B , and unload the cargo. But finding the solution can be difficult because the average branching factor is huge: each of the 50 planes can fly to 9 other airports, and each of the 200 packages can be either unloaded (if it is loaded), or loaded into any plane at its airport (if it is unloaded). On average, let's say there are about 1000 possible actions, so the search tree up to the depth of the obvious solution has about 1000^{41} nodes. It is clear that a very accurate heuristic will be needed to make this kind of search efficient. We will discuss some possible heuristics after looking at backward search.

Backward state-space search

Backward state-space search was described briefly as part of bidirectional search in Chapter 3. We noted there that backward search can be difficult to implement when the goal states are described by a set of constraints rather than being listed explicitly. In particular, it is not always obvious how to generate a description of the possible **predecessors** of the set of goal states. We will see that the STRIPS representation makes this quite easy because sets of states can be described by the literals that must be true in those states.

RELEVANCE

The main advantage of backward search is that it allows us to consider only **relevant** actions. An action is relevant to a conjunctive goal if it achieves one of the conjuncts of the goal. For example, the goal in our 10-airport air cargo problem is to have 20 pieces of cargo at airport B , or more precisely,

$$At(C_1, B) \wedge At(C_2, B) \wedge \dots \wedge At(C_{20}, B) .$$

Now consider the conjunct $At(C_1, B)$. Working backwards, we can seek actions that have this as an effect. There is only one: $Unload(C_1, p, B)$, where plane p is unspecified.

Notice that there are many *irrelevant* actions that can also lead to a goal state. For example, we can fly an empty plane from *JFK* to *SFO*; this action reaches a goal state from a predecessor state in which the plane is at *JFK* and all the goal conjuncts are satisfied. A backward search that allows irrelevant actions will still be complete, but it will be much less efficient. If a solution exists, it will be found by a backward search that allows only relevant actions. The restriction to relevant actions means that backward search often has a much lower branching factor than forward search. For example, our air cargo problem has about 1000 actions leading forward from the initial state, but only 20 actions working backward from the goal.

REGRESSION

Searching backwards is sometimes called **regression** planning. The principal question in regression planning is this: what are the states from which applying a given action leads to the goal? Computing the description of these states is called **regressing** the goal through the action. To see how to do it, consider the air cargo example. We have the goal

$$At(C_1, B) \wedge At(C_2, B) \wedge \dots \wedge At(C_{20}, B)$$

and the relevant action $Unload(C_1, p, B)$, which achieves the first conjunct. The action will work only if its preconditions are satisfied. Therefore, any predecessor state must include these preconditions: $In(C_1, p) \wedge At(p, B)$. Moreover, the subgoal $At(C_1, B)$ should not be true in the predecessor state.⁴ Thus, the predecessor description is

$$In(C_1, p) \wedge At(p, B) \wedge At(C_2, B) \wedge \dots \wedge At(C_{20}, B) .$$

CONSISTENCY

In addition to insisting that actions achieve some desired literal, we must insist that the actions *not undo* any desired literals. An action that satisfies this restriction is called **consistent**. For example, the action $Load(C_2, p)$ would not be consistent with the current goal, because it would negate the literal $At(C_2, B)$.

Given definitions of relevance and consistency, we can describe the general process of constructing predecessors for backward search. Given a goal description G , let A be an action that is relevant and consistent. The corresponding predecessor is as follows:

- Any positive effects of A that appear in G are deleted.
- Each precondition literal of A is added, unless it already appears.

Any of the standard search algorithms can be used to carry out the search. Termination occurs when a predecessor description is generated that is satisfied by the initial state of the planning problem. In the first-order case, satisfaction might require a substitution for variables in the predecessor description. For example, the predecessor description in the preceding paragraph is satisfied by the initial state

$$In(C_1, P_{12}) \wedge At(P_{12}, B) \wedge At(C_2, B) \wedge \dots \wedge At(C_{20}, B)$$

with substitution $\{p/P_{12}\}$. The substitution must be applied to the actions leading from the state to the goal, producing the solution $[Unload(C_1, P_{12}, B)]$.

⁴ If the subgoal were true in the predecessor state, the action would still lead to a goal state. On the other hand, such actions are irrelevant because they do not *make* the goal true.

Heuristics for state-space search

It turns out that neither forward nor backward search is efficient without a good heuristic function. Recall from Chapter 4 that a heuristic function estimates the distance from a state to the goal; in STRIPS planning, the cost of each action is 1, so the distance is the number of actions. The basic idea is to look at the effects of the actions and at the goals that must be achieved and to guess how many actions are needed to achieve all the goals. Finding the exact number is NP hard, but it is possible to find reasonable estimates most of the time without too much computation. We might also be able to derive an **admissible** heuristic—one that does not overestimate. This could be used with A* search to find optimal solutions.

There are two approaches that can be tried. The first is to derive a **relaxed problem** from the given problem specification, as described in Chapter 4. The optimal solution cost for the relaxed problem—which we hope is very easy to solve—gives an admissible heuristic for the original problem. The second approach is to pretend that a pure divide-and-conquer algorithm will work. This is called the **subgoal independence** assumption: the cost of solving a conjunction of subgoals is approximated by the sum of the costs of solving each subgoal *independently*. The subgoal independence assumption can be optimistic or pessimistic. It is optimistic when there are negative interactions between the subplans for each subgoal—for example, when an action in one subplan deletes a goal achieved by another subplan. It is pessimistic, and therefore inadmissible, when subplans contain redundant actions—for instance, two actions that could be replaced by a single action in the merged plan.

SUBGOAL
INDEPENDENCE

Let us consider how to derive relaxed planning problems. Since explicit representations of preconditions and effects are available, the process will work by modifying those representations. (Compare this approach with search problems, where the successor function is a black box.) The simplest idea is to relax the problem by *removing all preconditions* from the actions. Then every action will always be applicable, and any literal can be achieved in one step (if there is an applicable action—if not, the goal is impossible). This almost implies that the number of steps required to solve a conjunction of goals is the number of unsatisfied goals—almost but not quite, because (1) there may be two actions, each of which deletes the goal literal achieved by the other, and (2) some action may achieve multiple goals. If we combine our relaxed problem with the subgoal independence assumption, both of these issues are assumed away and the resulting heuristic is exactly the number of unsatisfied goals.

In many cases, a more accurate heuristic is obtained by considering at least the positive interactions arising from actions that achieve multiple goals. First, we relax the problem further by *removing negative effects* (see Exercise 11.6). Then, we count the minimum number of actions required such that the union of those actions' positive effects satisfies the goal. For example, consider

$$\begin{aligned} &Goal(A \wedge B \wedge C) \\ &Action(X, EFFECT: A \wedge P) \\ &Action(Y, EFFECT: B \wedge C \wedge Q) \\ &Action(Z, EFFECT: B \wedge P \wedge Q). \end{aligned}$$

The minimal set cover of the goal $\{A, B, C\}$ is given by the actions $\{X, Y\}$, so the set cover heuristic returns a cost of 2. This improves on the subgoal independence assumption, which

gives a heuristic value of 3. There is one minor irritation: the set cover problem is NP-hard. A simple greedy set-covering algorithm is guaranteed to return a value that is within a factor of $\log n$ of the true minimum value, where n is the number of literals in the goal, and usually works much better than this in practice. Unfortunately, the greedy algorithm loses the guarantee of admissibility for the heuristic.

It is also possible to generate relaxed problems by removing negative effects without removing preconditions. That is, if an action has the effect $A \wedge \neg B$ in the original problem, it will have the effect A in the relaxed problem. This means that we need not worry about negative interactions between subplans, because no action can delete the literals achieved by another action. The solution cost of the resulting relaxed problem gives what is called the **empty-delete-list** heuristic. The heuristic is quite accurate, but computing it involves actually running a (simple) planning algorithm. In practice, the search in the relaxed problem is often fast enough that the cost is worthwhile.

EMPTY-DELETE-LIST

The heuristics described here can be used in either the progression or the regression direction. At the time of writing, progression planners using the empty-delete-list heuristic hold the lead. That is likely to change as new heuristics and new search techniques are explored. Since planning is exponentially hard,⁵ no algorithm will be efficient for all problems, but many practical problems can be solved with the heuristic methods in this chapter—far more than could be solved just a few years ago.